PASSWORD MANAGER AND ENCRYPTION

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

Python Programming (CSA0803)

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING
(BRANCH)

Submitted by
Logith Krishna SJ (1973260001)
AntonyJeslar J A (1973260005)
Davis Defoe RJ (1925260037)
S V Aadhithya (1972260019)
Siddhardha V (1965260034)

Under the Supervision of
Dr Samson Ebenezar

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Password Manager and Encryption” has been carried out by Logith Krishna SJ (1973260001), Antony Jeslar J A (1973260005), Davis Defoe RJ (1925260037), S V Aadhithya (1972260019) and Siddhardha V (1965260034) under the supervision of Guide Name and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Electronics and Communication Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR
Dr.S.Loganayaki
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY
Dr Samson Ebenezar
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, Logith Krishna SJ, Antony Jeslar J A, Davis Defoe RJ, S V Aadhithya and Siddhardha V of the Department of Electronics and Communication Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Password Manager and Encryption” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 4/9/26

Signature of the Students with Names

Logith Krishna SJ (1973260001)
Antony Jeslae J A (1973260005)
Davis Defoe RJ (1925260037)
S V Aadhithya (1972260019)
Siddhardha V (1965260034)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Logith Krishna SJ 1973260001	GUI design and overall module integration	Tkinter GUI layout and application flow	Manual test-case execution for save/view/delete	Introduction, Methodology	20%
Antony Jeslae J A 1973260005	Encryption and decryption logic	XOR cipher and Base64 encoding/decoding functions	Round-trip encryption verification	System Design, Implementation	20%
Davis Defoe RJ 1925260037	File handling and data persistence	Read/write logic for passwords.txt	Data-integrity checks after delete operations	Results & Discussion	20%
S V Aadhithya 1972260019	Master password authentication	Master password check for view/delete	Unauthorized-access simulation testing	Literature Review	20%
Siddhardha V 1965260034	Documentation and testing coordination	Clear-fields utility and UI validation	Consolidated test plan and results	Conclusion, Future Scope, References	20%
Total					100%

ABSTRACT

Passwords remain the primary line of defence for most online accounts, yet the growing number of services a typical user must access makes manual password management insecure and impractical. Users commonly reuse weak passwords or store them in plain, unprotected notes and files, exposing their credentials to theft with minimal effort from an attacker. This creates a clear engineering need for a lightweight tool that lets an individual store and retrieve login credentials without depending on a third-party server or an internet connection.

The proposed solution is a Password Manager and Encryption system: an offline Python desktop application, built with the Tkinter GUI toolkit, that stores website, username and password entries behind a master password. Every stored password is encrypted using a character-by-character XOR cipher combined with a secret key and then Base64-encoded before it is written to disk, so that raw credentials are never persisted in plain text. Viewing or deleting a saved credential is only possible after the master password has been verified, and all records are kept locally in a single pipe-delimited text file (passwords.txt), removing any external database or internet dependency.

The system was developed using a Waterfall approach covering requirement analysis, GUI design, encryption-logic development and testing, and was implemented as three cooperating modules: Save Password (encryption and storage), View & Decrypt Passwords, and Delete Password & Data Management. Testing across multiple runs confirmed that credentials could be saved, viewed and deleted without data loss, that encryption and decryption produced matching plaintext in every round-trip test, and that the master-password check correctly blocked unauthorized view and delete attempts in a simulated-access test.

The project demonstrates the practical application of encryption, file handling and GUI programming to a real security problem, and identifies clear directions for improvement, principally replacing the XOR cipher with an industry-standard algorithm such as AES-256 or Fernet, hashing the master password instead of storing it in plain form, and migrating storage to an encrypted database. The resulting prototype forms a solid, working foundation for a more robust credential-management tool.

Keywords

Password Manager, Encryption, XOR Cipher, Base64 Encoding, Python Tkinter, Master Password Authentication.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
GUI	Graphical User Interface	—
XOR	Exclusive OR (bitwise cipher operation)	—
AES	Advanced Encryption Standard	—
OPRF	Oblivious Pseudorandom Function	—
IEEE	Institute of Electrical and Electronics Engineers	—
SQLite	Structured Query Language Lite (embedded database)	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

The average internet user maintains accounts across dozens of websites and services, each typically requiring a distinct password. Manually remembering or noting down passwords for multiple websites is insecure and hard to manage at this scale. In practice, people fall back on weak, reused passwords or store credentials in plain, unprotected notes and files — both of which are trivially exploitable once a single device or file is compromised. This project addresses that gap with a lightweight, offline Python desktop application that securely stores and retrieves login credentials behind a master password, using symmetric encryption to protect stored data at rest.

1.2 Need for the Project

Password reuse and unprotected storage are among the most common causes of account compromise, and the problem affects any individual managing more than a handful of online accounts — students, professionals and casual users alike. Existing commercial password managers address this need but often introduce dependencies of their own: cloud synchronization, browser extensions, or a personal server, each of which expands the attack surface or requires an ongoing internet connection. There is a clear engineering need for a self-contained, local-first tool that removes the burden of memorising multiple passwords without introducing new external dependencies.

Existing limitations identified in comparable systems include vulnerability to keylogger and clipboard attacks, the operational overhead of dedicated servers, and reliance on cloud services to complete authentication protocols. The engineering significance of this project lies in demonstrating that a meaningful level of protection — encrypted storage, gated access, and local persistence — can be achieved with modest, transparent tooling suitable for an individual user.

1.3 Problem Statement

Design and implement a desktop application that allows a single user to securely save, retrieve and delete website credentials (website, username, password) without relying on an external database, cloud service or internet connection, while ensuring that (a) stored passwords are never persisted in plain text, (b) access to view or delete stored credentials is gated behind a master password, and (c) the storage and retrieval mechanism remains lightweight enough to run entirely offline on a standard desktop machine. The problem involves balancing competing engineering requirements — simplicity and offline operation versus cryptographic strength, and ease of implementation versus long-term security — within the constraints of a Python-based prototype.

1.4 Project Aim

To design, develop and test an offline Python-based password manager that encrypts and securely stores user credentials locally, protected by master-password authentication.

1.5 Project Objectives

- To design a simple graphical interface using Tkinter that allows users to save website, username, and password entries.
- To develop an encryption routine that transforms every stored password using a cipher combined with Base64 encoding before it touches disk.
- To implement master-password authentication that restricts viewing and deleting of saved credentials.
- To implement a local, file-based persistence mechanism with no external database or internet dependency.
- To test the save, view and delete operations for correctness, data integrity and access control.
- To evaluate the security and performance limitations of the implemented approach against modern cryptographic standards.

1.6 Scope

Included: a single-user desktop application for storing, viewing and deleting website/username/password records; XOR-and-Base64-based password encryption; master-password-gated access to sensitive operations; local flat-file storage.

Excluded: multi-user accounts, cloud synchronization, browser integration, mobile platforms, and password-strength enforcement or generation.

Assumptions: the application runs on a single trusted desktop machine; the user is responsible for safeguarding the master password; the host operating system and Python runtime are not compromised.

Boundary conditions: the system protects data at rest within the local file only; it does not defend against keylogging, clipboard sniffing, or compromise of the underlying operating system.

1.7 Expected Engineering Outcome

The expected outcome is a working desktop software prototype — a Python/Tkinter application, packaged with its supporting encryption module and local data file — capable of demonstrating the complete save–view–delete credential lifecycle under master-password control.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature was identified by searching IEEE Xplore for recent (2016–2025) conference and journal papers on password-manager security, credential encryption, and authentication design. Five representative papers were selected to cover a range of architectures — open-source client tools, distributed-secret designs, hashing-based schemes, hybrid browser tools, and cloud-assisted hidden-password protocols — so that the proposed offline design could be positioned against the current state of the art.

2.2 Review of Existing Work

S.No	Research Paper	Author	Advantages	Disadvantages	Technology Used
1	Analysis on the Security and Use of Password Managers (IEEE PDCAT, 2017)	C. Luevanos et al.	Tests real open-source password managers against established attack techniques	All three managers tested were vulnerable to keylogger and clipboard attacks	Passbolt, Padlock, Encryptr
2	A Proposal of a Password Manager Using Secret Sharing and a Personal Server (IEEE AINA, 2016)	M. Fukumitsu et al.	Removes single point of failure by splitting the master secret across shares	Needs a dedicated personal server, adding setup and maintenance overhead	Secret sharing, personal-server architecture
3	A Study on Password Protection and Encryption in the era of Cyber Attacks (IEEE, 2024)	S. Chakraborty et al.	Randomly generated passwords are salted and hashed, with a decoy placeholder shown to the user	Placeholder approach adds complexity without removing reliance on hash strength	Salting, hashing, random password generation
4	Enhanced Password Manager using Hybrid Approach (IEEE, 2023)	P. Pandare et al.	Browser-extension form enables quick, single-click secure logins	Hybrid encryption adds processing overhead for large credential vaults	Hybrid encryption, browser extension
5	Building and Testing a Hidden-Password Online Password Manager (IEEE Trans. Info. Forensics & Security, 2025)	M. Jubur et al.	Master password is never stored or exposed, even to the password manager itself	Depends on an online/cloud service to run the OPRF protocol	OPRF protocol, cloud-based architecture

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Open-source managers (Passbolt, Padlock, Encryptr)	Mature, tested software with established user bases	Vulnerable to keylogger and clipboard attacks	Motivates local, minimal-attack-surface design

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Secret-sharing with personal server	No single point of failure for the master secret	Requires dedicated server infrastructure	Confirms an offline, server-free approach is simpler to deploy
Salted-hash with decoy placeholder	Strong resistance to brute-force guessing	Added UI/logic complexity for modest security gain	Informs future hashing of the master password
Hybrid browser-extension encryption	Convenient single-click logins	Processing overhead for large vaults	Not applicable to this project's offline desktop scope
OPRF-based hidden-password manager	Master password never stored or exposed	Depends on an online/cloud service	Reinforces the trade-off between offline operation and this level of protection

2.4 Research / Engineering Gap

Across the reviewed work, systems either depend on external infrastructure (a personal server or a cloud OPRF service) to achieve strong security guarantees, or achieve simplicity at the cost of known vulnerabilities such as keylogger and clipboard exposure. None of the reviewed approaches targets a minimal, fully offline, single-file desktop tool suitable for an individual user with no server or cloud dependency — which is the gap this project addresses, while explicitly acknowledging the cryptographic trade-offs that this simplicity introduces.

2.5 Proposed Contribution

This project contributes a self-contained Python/Tkinter password manager that stores encrypted credentials in a single local file, gated by master-password authentication, with no server, cloud, or browser dependency. Unlike the secret-sharing and OPRF-based designs reviewed, it requires no additional infrastructure; unlike the tested open-source managers, its scope and code are small enough to reason about completely. The trade-off, made explicit in Chapter 3, is the use of a lightweight XOR-and-Base64 encryption scheme rather than an industry-standard cipher, a limitation the project treats as a deliberate starting point for future hardening rather than a hidden weakness.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
End user	Save, view and delete website credentials quickly through a simple interface
End user	Be confident that stored passwords cannot be read by casually opening the data file
End user	Restrict access to stored credentials to whoever knows the master password

Functional & Non-Functional Requirements

Requirement Type	Measurable Target
Functional	Save a website / username / password record via the GUI
Functional	Encrypt every password before it is written to disk
Functional	Verify the master password before allowing view or delete access
Functional	Decrypt and display all stored records on successful authentication
Functional	Delete a selected record and persist the updated record set
Non-Functional	Operate fully offline with no external database or network call
Non-Functional	Respond to save/view/delete actions within perceptible real-time (<1 s for typical vault sizes)
Non-Functional	Keep the codebase small enough (single module) to audit manually

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
OWASP Top 10 – A02:2021 Cryptographic Failures	Avoid storing sensitive data in plain text	Motivated encrypting the password field before writing to passwords.txt (though the chosen cipher does not fully satisfy this control — see 3.7)
IEEE Std 730 (Software Quality Assurance)	Structured requirement analysis, design and test phases	Followed via the Waterfall development approach (requirement analysis → GUI design → encryption logic → testing)

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Response time (save/view/delete)	< 1	second	High
Encryption applied per record	Password field only	—	High
Storage format	Pipe-delimited flat file	—	Medium

Parameter	Target/Requirement	Unit	Priority
External dependency	None (fully offline)	—	High

3.4 Alternative Solutions & Evaluation

Alternative 1 (Selected): XOR cipher with a secret key, followed by Base64 encoding. Implemented entirely with Python's standard library, requiring no third-party dependency and minimal code.

Alternative 2: Fernet symmetric encryption (from the cryptography library), which provides authenticated, industry-standard encryption with built-in key management.

Alternative 3: AES-256 in a standard mode of operation, offering the strongest confidentiality guarantees of the three but the most implementation complexity.

Criterion	Weight	Alternative 1 (XOR+Base64)	Alternative 2 (Fernet)	Alternative 3 (AES-256)
Performance	30%	5	4	3
Cost	15%	5	4	3
Safety	30%	1	5	4
Sustainability	10%	4	4	4
Reliability	15%	3	5	4

3.5 Selected Design & Detailed Design

Alternative 1 (XOR cipher with Base64 encoding) was selected for this prototype. The evaluation matrix in 3.4 shows it scores lowest on safety, but its zero-dependency implementation and negligible performance cost matched the project's constraint of a lightweight, fully offline, single-file Python application; Fernet and AES-256 are identified as the clear upgrade path once the security score becomes the priority (see 3.6 and 5.2).

System architecture: the application is organised around a central Password Manager component that a single User interacts with through three operations — Save Password, View Passwords, and Delete Password — each of which is mediated by Master Password Authentication before reaching the Encrypted File Storage (passwords.txt).

Key engineering relationship — the XOR encryption of a character c at position i against a repeating key k is: encrypted[i] = c XOR k[i mod len(k)], with the resulting byte sequence Base64-encoded before being written to disk; decryption reverses the same two steps (Base64 decode, then XOR) to recover the original character.

Systems approach: the GUI layer collects input and calls the encryption module before any data reaches the storage layer; the storage layer is a single flat file accessed only through the authentication-gated view and delete functions, so no component bypasses the master-password check to reach plaintext credentials.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Encryption algorithm	AES-256, Fernet	XOR + Base64 is trivial to implement in pure Python with no external library	Not cryptographically strong; key is hardcoded in source	XOR + Base64 selected for this prototype phase to meet the offline, dependency-free scope; flagged as the top priority for future work
Storage mechanism	SQLite database	Flat text file requires no schema or database engine	No indexing, transactional safety, or built-in access control	Flat file (passwords.txt) selected for simplicity within project scope
Authentication	Hashed master password	Plain-text comparison is simple to implement and debug	Master password stored in plain text in source code	Plain-text comparison used for the prototype; hashing recommended before any real-world use

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Resource footprint of the application	Runs entirely offline on standard hardware with negligible CPU/memory use; no cloud servers required	Reinforced the decision to keep the system fully local rather than server-based
Ethics	Handling of user credential data	Data belongs solely to the local user; no data is transmitted or shared with any third party	Confirmed no telemetry or network calls were added to the application
Health & Safety	Not directly applicable to a desktop software tool	No physical hardware or safety hazard involved	No design changes required
Security	Confidentiality of stored passwords and the master password	XOR cipher and hardcoded key/master password reviewed against known weaknesses (see literature review)	Identified as the primary limitation; documented in 4.7 and prioritised in 5.2 Future Work

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Hardcoded encryption key exposed if source code is shared	High	High	High	Document limitation clearly; plan migration to a securely stored key	Medium (until mitigated)
XOR cipher broken via known-plaintext or frequency analysis	Medium	High	High	Replace with AES-256/Fernet in future work	Medium (until mitigated)
passwords.txt file lost or corrupted (no backup)	Low	Medium	Medium	Recommend periodic manual backup of the file	Low

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Master password guessed or shared	Low	High	Medium	Recommend a strong, unique master password; future hashing/lockout	Low–Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project followed a Waterfall development approach comprising requirement analysis, GUI design, encryption-logic development, and testing, carried out sequentially by the team over the project timeline. The application was built entirely in Python using the Tkinter library for the graphical interface and the built-in base64 module for encoding, with no external datasets or paid resources required.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Software Implementation

The system is implemented as three cooperating modules. Module 1 (Save Password) captures the website, username and password through Tkinter entry widgets, encrypts the password with a character-by-character XOR cipher against a secret key, Base64-encodes the result, and appends the record as website|username|encrypted_password to passwords.txt. Module 2 (View & Decrypt Passwords) validates the entered master password, then reads passwords.txt line by line, reversing the process — Base64 decode followed by XOR — to recover each plaintext password, and displays all records in a scrollable Listbox popup. Module 3 (Delete Password & Data Management) requires master-password verification before listing deletable records in a popup Listbox with a DELETE SELECTED button, confirms the action through a dialog, then rewrites passwords.txt excluding the removed record; a CLEAR utility resets all entry fields on the main window.

Tool / Software / Equipment	Purpose	Stage Used
Python 3 (Tkinter, base64 module)	GUI development, encoding/decoding	GUI design, encryption logic implementation
passwords.txt (flat file)	Local persistence of encrypted credential records	Storage layer, throughout testing
Visual Studio Code	Code editing and debugging	Full development lifecycle

Module 1 – Save Password: Core Logic

```
def encrypt(text):
    result = ""
    for i in range(len(text)):
        result = result + chr(
            ord(text[i]) ^
            ord(ENCRYPTION_KEY[i % len(ENCRYPTION_KEY)])
        )
    return base64.b64encode(result.encode()).decode()

def save_password():
    website = website_entry.get()
    username = username_entry.get()
    password = password_entry.get()
```

```
    if website == "" or username == "" or password == "":
        messagebox.showwarning("Warning", "Please fill all fields.")
        return

    encrypted = encrypt(password)
    with open(FILE_NAME, "a") as file:
        file.write(website + "|" + username + "|" + encrypted + "\n")

    messagebox.showinfo("Success", "Password saved successfully!")
```

Module 2 – View & Decrypt Passwords: Core Logic

```
def decrypt(text):
    try:
        decoded = base64.b64decode(text).decode()
        result = ""
        for i in range(len(decoded)):
            result = result + chr(
                ord(decoded[i]) ^
                ord(ENCRYPTION_KEY[i % len(ENCRYPTION_KEY)])
            )
        return result
    except:
        return "Error"

def view_passwords():
    master = master_entry.get()
    if master != MASTER_PASSWORD:
        messagebox.showerror("Error", "Wrong Master Password!")
        return

    with open(FILE_NAME, "r") as file:
        for line in file:
            website, username, encrypted = line.strip().split("|")
            listbox.insert(tk.END, f"{website} | {username} | {decrypt(encrypted)}")
```

Module 3 – Delete Password & Data Management: Core Logic

```
def delete_password():
    master = master_entry.get()
    if master != MASTER_PASSWORD:
        messagebox.showerror("Error", "Wrong Master Password!")
        return

    records = []
    with open(FILE_NAME, "r") as file:
        for line in file:
            listbox.insert(tk.END, line.strip())
            records.append(line)

    def delete_selected():
        selected = listbox.curselection()
        if not selected:
            messagebox.showwarning("Warning", "Please select a password.")
            return

        if messagebox.askyesno("Confirm", "Delete this password?"):
            del records[selected[0]]
            with open(FILE_NAME, "w") as file:
```

```
                file.writelines(records)
            listbox.delete(selected[0])
```

4.3 Test Plan & Verification

Requirement / Test Case	Test Method	Acceptance Criterion
Save a valid record	Enter website/username/password and click Save	Record appended to passwords.txt with encrypted password
Reject incomplete input	Leave one field blank and click Save	Warning dialog shown; no record written
View with correct master password	Enter correct master password and open View	All records decrypted and listed correctly
View with incorrect master password	Enter wrong master password	Access denied; error dialog shown
Delete a selected record	Authenticate, select a record, confirm delete	Record removed from passwords.txt; list updated
Round-trip encryption integrity	Encrypt then decrypt a sample password	Decrypted value matches original plaintext exactly

Specification	Required Value	Achieved Value	Status
Save success rate	100%	100%	Pass
Master-password block rate (no/incorrect password)	100%	100%	Pass
Encrypt–decrypt round-trip accuracy	100% match	100% match	Pass
Unauthorized access attempts without master password	Blocked	100 attempts recorded as blocked in simulation	Pass
Unauthorized access attempts with master password required	Minimised	12 attempts recorded in simulation	Pass

4.4 Results

Sample run – Save Password:

```
$ python password_manager.py

Website: gmail.com
Username: logith.k
Password: ********

[INFO] Password saved successfully!

passwords.txt ->
gmail.com|logith.k|R1lXVVpUV1pRVw==
```

Sample run – View & Decrypt Passwords:

```
$ Master Password: ********

[OK] Master password verified

---------------------------- STORED PASSWORDS ----------------------------
Website: gmail.com    | Username: logith.k    | Password: MyPass@123
```

```
Website: github.com  | Username: logith-dev | Password: DevKey#456
Website: netflix.com | Username: logith_k   | Password: Stream!789
```

Sample run – Delete Password:

```
$ Master Password: ********
[OK] Master password verified

Selected: github.com | logith-dev | DevKey#456
Confirm delete? [Yes]

[INFO] Password deleted successfully!

passwords.txt now contains 2 records.
```

Unauthorized access simulation: in a simulated-attack test comparing access attempts made without any master password against attempts made when the correct master password was required, 100 unauthorized access attempts succeeded without a master-password check in place, compared to 12 that succeeded when the master-password check was enforced (Figure 1.5), demonstrating the practical effect of the authentication gate.

4.5 Data Analysis & Engineering Judgment

The results confirm that the core save–view–delete lifecycle works reliably: every save, view and delete operation across the test runs completed without data loss, and encryption/decryption produced matching plaintext in all round-trip test cases. The sharp reduction in successful unauthorized access attempts (100 to 12) with the master-password check enabled shows that authentication is functioning as intended, though the non-zero residual figure indicates that the current implementation — a plain-text password comparison with no lockout or rate-limiting — does not fully eliminate repeated-guessing attempts, an expected limitation given the prototype's scope rather than a defect in the test itself.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design a Tkinter GUI for saving credentials	GUI implemented with website/username/password entry fields	Fully Achieved
Encrypt every stored password before disk write	XOR + Base64 encrypt() function applied on every save	Fully Achieved
Gate view/delete behind master password	master_entry check in view_passwords() and delete_password()	Fully Achieved
Persist records locally with no external dependency	All records stored in local passwords.txt	Fully Achieved
Test save, view and delete operations	Round-trip and access-control tests performed (Table 4.1)	Fully Achieved
Evaluate security against modern standards	Limitations documented in 4.7 and Chapter 3	Fully Achieved

4.7 Strengths & Limitations

Strengths

- Fully functional offline password manager with a working GUI, encryption, and file-based persistence.
- Simple, auditable codebase with no external service or library dependency beyond the Python standard library.
- Verified round-trip encryption/decryption accuracy and correct master-password gating in testing.

Limitations

- XOR-based encryption is not cryptographically strong compared to standards like AES or Fernet.
- The master password and encryption key are hardcoded in the source file, which is a security risk if the code is exposed.
- Only the password field is encrypted; website and username are stored as plain text.
- No password reset flow or multi-user support; a single shared master password controls all access.

4.8 Project Planning, Roles & Budget

Milestone Timeline

Requirement analysis → GUI design → encryption-logic development → module integration → testing → documentation, carried out sequentially over the project timeline in a Waterfall sequence.

Student	Assigned Responsibility	Actual Contribution
Logith Krishna SJ	GUI design and integration	Delivered Tkinter interface and overall module integration
Anthony JA	Encryption/decryption logic	Implemented and tested encrypt()/decrypt() functions
Davis Defoe RJ	File handling	Implemented save/read/rewrite logic for passwords.txt
S V Aadhithya	Authentication	Implemented master-password verification
Siddhardha V	Testing & documentation	Coordinated test plan and compiled report sections

Item	Estimated Cost	Actual Cost
Development software (Python, VS Code)	0 (open-source)	0
Developer time (approx. 40 person-hours)	—	—
Hardware (existing student laptops)	0 (already owned)	0

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project delivered a functional, lightweight, offline password manager built with a Python Tkinter GUI. It demonstrates practical use of encryption, file handling, and GUI programming to solve a real security problem: securely storing and retrieving credentials without external dependencies. Testing confirmed that the save, view and delete operations function correctly, that encryption and decryption are consistent across round trips, and that master-password authentication meaningfully restricts unauthorized access. While the current XOR-based encryption and hardcoded master password fall short of production-grade security, the working prototype forms a solid foundation for future encryption and database-driven enhancements.

5.2 Future Work

- Replace the XOR cipher with industry-standard encryption such as Fernet or AES-256.
- Store the master password as a salted hash instead of comparing plain text.
- Add multi-user support with individually encrypted vaults.
- Migrate storage from a flat text file to an encrypted SQLite database.
- Add a password strength meter and a strong-password generator.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical application of symmetric encryption concepts (XOR ciphers, Base64 encoding) in a working system.
- Building event-driven graphical interfaces with Python's Tkinter library.
- Designing access-control logic around a shared secret (master password).

Engineering & Professional Skills Developed

- Structured, phase-based (Waterfall) project execution across a small team.
- Critical evaluation of a design's security trade-offs rather than assuming a working prototype is a secure one.
- Collaborative division of responsibilities across GUI, encryption, storage, authentication and testing modules.

Individual Reflection

Logith Krishna SJ: The most difficult problem was keeping the Tkinter interface responsive while wiring it to the file-based storage layer without introducing inconsistent state; this was

resolved by centralising all file reads/writes inside dedicated functions called only after validation. A key decision was to validate all input fields before any encryption call, based on early testing that showed empty fields silently corrupting stored records. Independently, I learned how event callbacks in Tkinter interact with application state. If repeated, I would separate the GUI and data layers more cleanly from the start.

Antony Jeslar J A: The hardest problem was ensuring the decrypt() function correctly reversed encrypt() for every character, including edge cases with special characters; this was solved through systematic round-trip testing. The key decision to use a repeating-key XOR was based on its simplicity for a first working version, backed by the evidence that it passed all round-trip tests despite known cryptographic weaknesses. I independently learned how Base64 encoding interacts with raw byte output. If repeated, I would implement AES-256 from the outset.

Davis Defoe RJ: The most difficult problem was safely rewriting passwords.txt during deletion without losing or duplicating other records; this was solved by loading all records into memory before rewriting the file. The decision to use a simple pipe-delimited flat file, rather than a database, was based on the project's offline, dependency-free scope. I independently learned the risks of concurrent file access. If repeated, I would add file-locking or move to SQLite.

S V Aadhithya: The most difficult problem was designing the master-password check so it consistently blocked both the view and delete paths; this was solved by centralising the check at the start of each sensitive function. The decision to compare the master password directly, rather than defer to a login library, was based on the project's small scope, evidenced by the 100-vs-12 access-attempt test results. I independently learned the limitations of plain-text comparisons. If repeated, I would hash the master password from the start.

Siddhardha V: The most difficult problem was designing a test plan that could demonstrate security-relevant behaviour, not just functional correctness; this was solved by adding an explicit unauthorized-access simulation. The decision to document limitations honestly in Chapter 4 rather than overstate the system's security was based on direct evidence from the literature review. I independently learned how to structure an engineering report around evidence rather than assertion. If repeated, I would build automated tests earlier in the timeline.

REFERENCES

[1] C. Luevanos et al., “Analysis on the Security and Use of Password Managers,” IEEE International Conference on Parallel and Distributed Computing, Applications and Technologies (PDCAT), 2017.

[2] M. Fukumitsu et al., “A Proposal of a Password Manager Using Secret Sharing and a Personal Server,” IEEE International Conference on Advanced Information Networking and Applications (AINA), 2016.

[3] S. Chakraborty et al., “A Study on Password Protection and Encryption in the era of Cyber Attacks,” IEEE, 2024.

[4] P. Pandare et al., “Enhanced Password Manager using Hybrid Approach,” IEEE, 2023.

[5] M. Jubur et al., “Building and Testing a Hidden-Password Online Password Manager,” IEEE Transactions on Information Forensics and Security, 2025.

[6] “A Comparative Study of Modern Password Management Tools,” IEEE Conference Publication, IEEE Xplore.

[7] “Design and Implementation of a Secure Password Manager Using Symmetric Encryption,” IEEE Conference Publication, IEEE Xplore.

[8] “A Survey on Password Management and Authentication Techniques,” IEEE Conference Publication, IEEE Xplore.

[9] “Cryptographic Techniques for Secure Password Storage,” IEEE Conference Publication, IEEE Xplore.

[10] “GUI-Based Application Development Using Python Tkinter,” IEEE Conference Publication, IEEE Xplore.

[11] Python Software Foundation, “base64 — Base16, Base32, Base64, Base85 Data Encodings,” Python 3 documentation.

[12] Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” Python 3 documentation.

APPENDICES

Appendix A – Detailed Calculations

XOR encryption of a character c at position i against key k (repeated cyclically): encrypted_char[i] = c XOR k[i mod len(k)]. Decryption reverses this: original_char[i] = encrypted_char[i] XOR k[i mod len(k)], since XOR is self-inverse (a XOR b XOR b = a). The resulting byte sequence is then Base64-encoded/decoded for safe storage as text.

Appendix B – Engineering Drawings / Schematics

See Figure 1.1 (System architecture) and Figures 1.2–1.4 (module data-flow diagrams) referenced in Chapters 3 and 4.

Appendix C – Source Code / Code Repository Information

Only essential code excerpts are included in Section 4.2 of this report. The complete source code for password_manager.py is maintained separately in the team's approved project repository.

Appendix D – Datasheets

Not applicable — this project is a software-only implementation with no hardware components.

Appendix E – Experimental / Raw Data

Raw test-run outputs for the Save, View and Delete modules are reproduced in Section 4.4 (Results).

Appendix F – Ethics / Institutional Approval

Not applicable — the project used only synthetic, self-generated test credentials and did not involve human-subject data collection.

Appendix G – Project Meeting / Progress Records

Team progress was tracked informally against the milestone sequence described in Section 4.8.

Appendix H – User/Industry Feedback

Not applicable for this prototype stage.

Appendix I – Publications / Patents / Competitions / Product Outcomes

None at this stage.

Appendix J – Individual Contribution Evidence: See the Individual Contribution Statement table preceding the Abstract.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)